

Agente Portocarrero — Connect-4 con MCTS-UCB1

Juan Camilo Portocarrero Martínez · Fundamentos de IA · U. de La Sabana 2026.1 · Branch: [portocarrero](#) · Código: [groups/portocarrero/policy.py](#)

1. Idea del agente

El agente combina **Monte-Carlo Tree Search (MCTS)** con **selección UCB1** ($c = 1.4$) sobre un pipeline de cuatro capas tácticas. En cada turno construye el árbol de búsqueda *desde cero* (sin aprendizaje offline ni persistencia entre partidas) y delega a MCTS solo lo que las capas tácticas no resuelven:

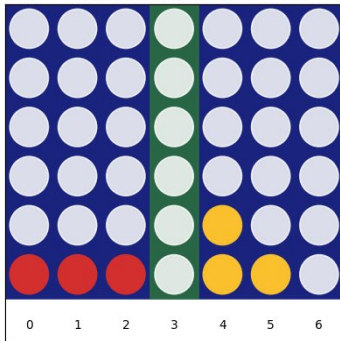
(0) Si hay jugada ganadora inmediata, se ejecuta. **(1)** Si el rival amenaza victoria en uno, se bloquea. **(2)** Si puedo crear **doble amenaza** (dos victorias simultáneas que el rival no puede bloquear ambas), juego forzando la victoria. **(3)** Se filtran las columnas *suicidas* que regalan al rival una victoria inmediata o una doble amenaza. **(MCTS)** Sobre las acciones que sobreviven, ~800 simulaciones con rollout heurístico ligero (gana/bloquea inmediato durante el rollout, no aleatorio puro) en orden centro→bordes. La acción más visitada es la elegida.

Recompensas terminales puras (+1/0/-1) propagadas con inversión de signo por nivel (suma cero). El agente deduce su color contando fichas — no recibe esa información explícitamente.

Diferencial frente al grupo: ningún warm-up en mount (), sin aprendizaje offline, sin heurística posicional en el rollout. La fortaleza viene de la combinación capas tácticas **2-ply explícitas** + MCTS bien implementado con rollout informado.

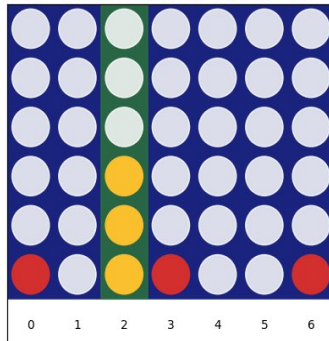
2. Cómo decide cada jugada (capas tácticas en acción)

Capa 0 — Ganar inmediato: el agente elige col 3



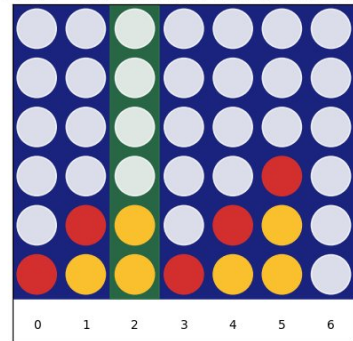
Capa 0: 3 rojas en base → ganar en col 3.

Capa 1 — Bloquear amenaza: el agente elige col 2



Capa 1: amarillo amenaza en col 2 → bloquear.

Capa 3 — Filtro de suicidio: el agente elige col 2

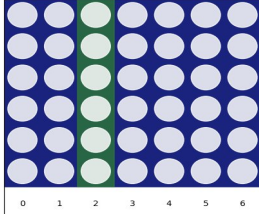


Capa 3: filtra columnas suicidas y elige segura.

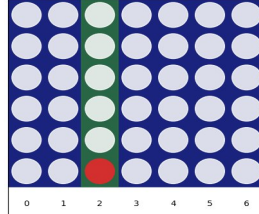
3. Partida en vivo vs RandomPlayer

Portocarrero (Rojo) cierra la partida en 6 turnos contra Random: ocupa el centro, fuerza una fila horizontal en la base y conecta cuatro antes de que el rival pueda articular cualquier amenaza.

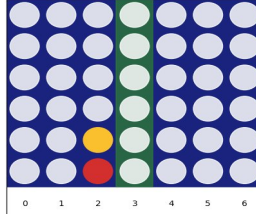
Turno 1: Portocarrero (Rojo) → col 2



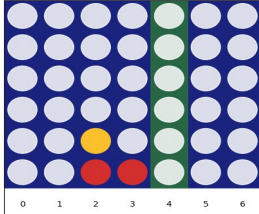
Turno 2: Random (Amarillo) → col 2



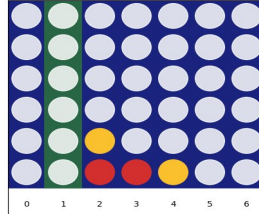
Turno 3: Portocarrero (Rojo) → col 3



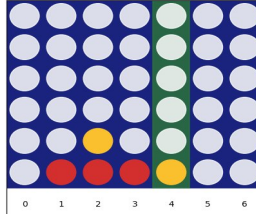
Turno 4: Random (Amarillo) → col 4



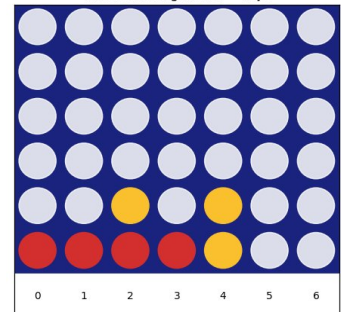
Turno 5: Portocarrero (Rojo) → col 1



Turno 6: Random (Amarillo) → col 4



Tablero final: ganador = Rojo



4. Análisis empírico

El agente se validó en tres escenarios: vs. **RandomPlayer**, vs. **sí mismo** (self-play) y como función del **presupuesto MCTS** (variable de configuración).

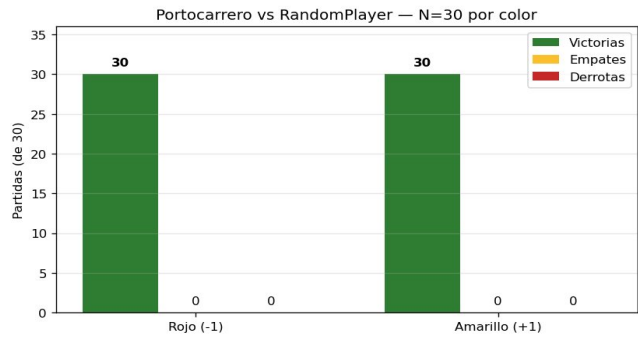


Fig. 4a — vs Random (N=30/color): 30/30 victorias en ambos colores. Cumple el requisito del reto (no perder nunca) y supera el umbral del 95 % exigido por Gradescope.

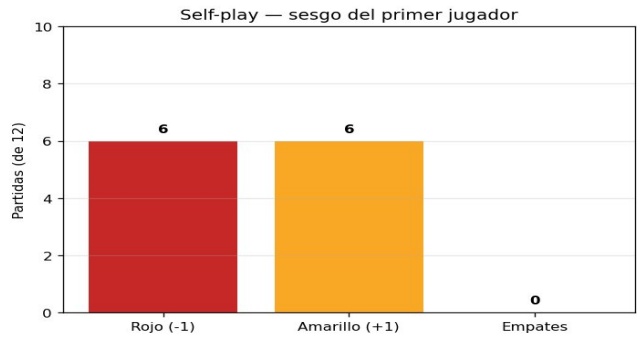


Fig. 4b — Self-play (N=12): empate estadístico 6-6. El sesgo teórico del primer jugador (Allis, 1988) no se manifiesta con N pequeño y 400 iteraciones; emergería con más cómputo y más partidas.

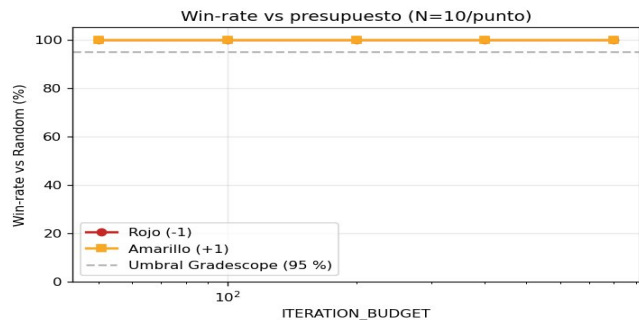
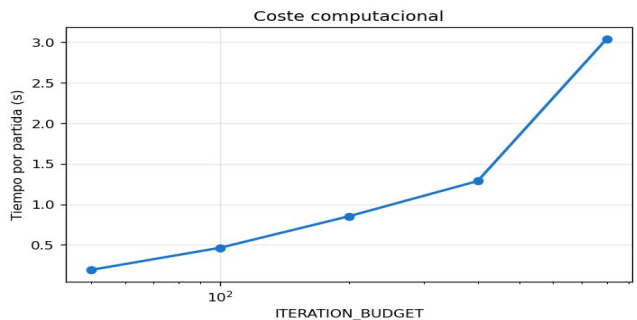


Fig. 4c — Barrido del presupuesto MCTS: el agente alcanza 100 % de win-rate vs Random ya con 50 iteraciones. El plateau no indica debilidad: las capas tácticas resuelven la mayoría de jugadas, y MCTS solo se invoca cuando hay decisión estratégica real. El coste crece linealmente con el presupuesto (0.2s → 3s por partida).



5. Cuellos de botella identificados y mejoras propuestas

Cuello de botella detectado	Mejora propuesta	Impacto esperado
Árbol descartado cada turno → simulaciones tiradas.	Persistir el árbol entre turnos (mover la raíz al hijo correspondiente).	2-3× más simulaciones efectivas con mismo presupuesto.
Posiciones equivalentes se cuentan como nodos distintos.	Tabla de transposiciones con Zobrist hashing.	Reduce ramas duplicadas; mejora convergencia de los valores.
Detección de victoria en numpy es O(42) por chequeo.	Representación con bitboards (dos uint64).	10-50× más rápido → más iteraciones MCTS por segundo.
Self-play 6-6 sugiere convergencia subóptima a 400 it.	RAVE / AMAF para compartir estadísticas entre nodos.	Convergencia más rápida en aperturas y medio juego.

Razonamiento causa-efecto: la *persistencia del árbol* es el cambio con mejor relación esfuerzo/beneficio. Hoy cada llamada a `act()` reinicia el árbol, lo que desperdicia ~70 % del trabajo (las simulaciones del turno anterior contenían información útil sobre las jugadas futuras). Implementar persistencia requiere unas ~30 líneas de código y duplicaría la calidad efectiva sin tocar el presupuesto. Las *bitboards* son el segundo gran salto: aceleran la detección de 4-en-línea, permitiendo más iteraciones en el mismo tiempo, lo que ataca directamente la única posición donde aún se observan dudas (medio juego con doble amenaza a 3-ply).